Respuestas a la Guía del Scientific Workshop + Hipótesis y Protocolo Experimental

Artículo: **Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes (EVAPORATE)** · PVLDB Vol. 17, No. 2 (2023)

1. Identificación

1.1. Título del artículo, autores, año y publicación

- **Título:** *Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes* (Sistema **EVAPORATE**).
- **Autores:** Simran Arora, Brandon Yang*, Sabri Eyuboglu*, Avanika Narayan, Andrew Hojel (*Stanford University*), Immanuel Trummer (*Cornell University*) y Christopher Ré (*Stanford University*).
- **Año y Publicación:** 2023 · *Proceedings of the VLDB Endowment (PVLDB)*, Vol. 17, No. 2, págs. 92–105 (ISSN: `2150-8097`). Clasificación: **Revista JCR Q1** y **Congreso CORE A***.

1.2. Referencia bibliográfica y DOI

- **Referencia (IEEE/ACM):** S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023, doi: 10.14778/3626292.3626294.
- **DOI oficial:** https://doi.org/10.14778/3626292.3626294 · **PDF VLDB:** https://www.vldb.org/pvldb/vol17/p92-arora.pdf · **Código:** https://github.com/HazyResearch/evaporate

2. Motivación y problema

2.1. ¿Qué problema de investigación aborda?

Aborda la **generación automática de vistas relacionales estructuradas (tablas SQL) a partir de Data Lakes heterogéneos** compuestos por documentos semi-estructurados y no estructurados (HTML, PDF, TXT), sin datos de entrenamiento etiquetados ni esquema previo. Dada una colección de n documentos $D = \{d_1, \ldots, d_n\}$, descubre el esquema de atributos $A = \{a_1, \ldots, a_m\}$ y extrae los n registros $R = \{r_1, \ldots, r_n\}$.

2.2. ¿Por qué es relevante?

Porque el 80% de los datos empresariales son no estructurados y los sistemas previos fracasan ante el **Trilema Generalidad – Coste – Calidad**: los modelos Web/DOM (`DOM-LM`, `Ceres`) exigen entrenamiento supervisado y solo funcionan en nodos hoja HTML; los sistemas NLP abiertos (`OpenIE6`) colapsan en documentos largos (extrayendo 427 relaciones sin normalizar en un solo informe FDA); y aplicar un LLM documento a documento tiene un coste lineal $O(n)$ prohibitivo ($5.157 por 10k docs; $1,1M en Wikipedia).

3. Propuesta

3.1. ¿Cuál es la principal contribución?

Demostrar que en vez de procesar cada documento directamente con el LLM (`Evaporate-Direct`, coste $O(n)$), es más barato y más preciso usar el LLM para **sintetizar múltiples funciones extractoras en Python** sobre una muestra de $k = 10$ documentos y agregar sus salidas sin etiquetas humanas mediante el **Algoritmo 1 de Supervisión Débil (Weak Supervision)** (`Evaporate-Code+`, coste sublineal $O(m \cdot k)$), logrando ahorrar **110× en tokens** y mejorar la calidad en **+10,1 puntos Pair F1**.

3.2. ¿Qué método, arquitectura o técnica utilizan?

Opera en 3 etapas con 6 plantillas de prompt (más clustering previo **TF-IDF + K-Means** si hay múltiples fuentes mezcladas): (1) *Schema Synthesis* sobre $k = 10$ documentos priorizando atributos frecuentes; (2) *Diverse Code Synthesis* combinando prompts orientados a regex (`P_A`) y a librerías como `BeautifulSoup` (`P_B`); y (3) *Function Aggregation (Algoritmo 1)*, que estima la tasa de presencia e en D_eval (≤10 docs) para distinguir abstenciones de valores nulos, **filtra todas las funciones ruidosas con calidad empírica $\hat{a}_j \leq 0{,}5$** (Proposición 1), discretiza el texto en los *top-b buckets* y resuelve un modelo gráfico latente.

4. Evaluación y resultados

4.1. ¿Cómo validan su propuesta? ¿Con qué datos, experimentos o métricas?

Evalúan en **ClosedIE** (Text F1) y **OpenIE End-to-End** (Pair F1, tokens y coste $) sobre **16 benchmarks reales de 5 dominios y 3 formatos** (*SWDE Movies* 8 webs HTML, *SWDE Universities* 5 webs HTML, *Wikipedia NBA* HTML complejo, *Enron Emails* 500k TXT y *FDA 510(k)* PDFs médicos de ~20 págs.) con **4 LLMs** (`text-davinci-003` , `GPT-4` , `Claude-v1` , `Jurassic-2`).

4.2. ¿Cuáles son los principales resultados?

- **Reducción de coste de 110× (y 378× en tamaño real):** De $5.157 (258M tokens) en `Direct` a solo $39 (1,9M tokens) en `Code+` por cada 10.000 documentos, tardando 0,25 ms/doc en CPU.
- **Mejora de calidad (+10,1 Pair F1 / +18%):** `Code+` alcanza **66.7 Pair F1** frente a 56.6 de `Direct` y 42.8 de `Code` (+23,4 pts en Wiki NBA y +17,3 pts en FDA), y supera al SoTA supervisado (`DOM-LM`) en **+6% en OpenIE** y **+10% en ClosedIE** sin entrenar.

5. Análisis crítico, Hipótesis de Mejora y Cómo Medirla (Aportación del Grupo)

5.1. Crítica fundamentada: El cuello de botella del Recall (58,9%) y la caída en textos libres

En la Tabla 1 del paper detectamos una fuerte asimetría en `Evaporate-Code+` : logra una **Precisión muy alta (78,5%)** pero un **Recall bajo (58,9%)** (deja sin extraer el 41,1% de los valores reales). Además, en textos libres como *Enron Emails* (Tabla 3), `Code+` (86.9 F1) pierde **-6,9 puntos F1** frente a `Direct` (93.8 F1) porque cuando un documento presenta variaciones sintácticas o prosa libre, todas las funciones Python se abstienen y dejan la celda vacía.

Hipótesis Principal H_1 propuestas por el Grupo (*EVAPORATE-Hybrid*):
*"Si ejecutamos `Evaporate-Code+` en CPU como extractor primario, pero utilizamos la matriz de votos Ψ y la tasa de presencia e del Algoritmo 1 como un **router** para invocar selectivamente al LLM con esquema fijo (**Closed-Schema Fallback**) únicamente en las celdas (d_i, a_j) donde las funciones filtradas F' se abstienen y el atributo tiene alta presencia ($e_j > \tau$) o baja confianza posterior $P(\hat{y}|\Psi) < \theta$, entonces **aumentaremos significativamente el Recall y el Pair F1 global (recuperando la caída en Enron)** y mantendremos un ahorro >30× frente a `Evaporate-Direct` , sin sufrir sus alucinaciones de esquema."*
Hipótesis Secundaria H_2 (*EVAPORATE-Refine*): Re-promptear 1 sola vez en *D_eval* las funciones descartadas ($\hat{a}_j \leq 0{,}5$, que son el 51% del total) mostrando el contraejemplo donde fallaron elevará la tasa de supervivencia de funciones del 49% a >70% a coste constante $O(m \cdot k)$.

5.2. ¿Cómo vamos a medir si nuestra hipótesis mejora los resultados? (Protocolo Experimental)

1. **Variables Independientes:** Variar el presupuesto de fallback $\beta \in \{0\%, 5\%, 10\%, 20\%\}$ de celdas enrutadas al LLM (controlado por el umbral de confianza θ) y las rondas de refinamiento $r \in \{0, 1\}$ en *D_eval*.
2. **Baselines de Control:** Comparar contra (1) `Evaporate-Code+` original ($\beta = 0\%$), (2) `Evaporate-Direct` ($\beta = 100\%$), y (3) `Random-Fallback` (enviar el mismo $\beta\%$ de celdas al azar en vez de guiado por la abstención del Algoritmo 1).
3. **Métricas de Evaluación:** Medir **ΔRecall**, **ΔPair F1**, **Eficiencia Marginal (ΔF1 / Millón de Tokens)** y **Factor de Ahorro Neto** (Tokens Direct / Tokens Hybrid).

Benchmark (Resultados de `experimento_hipotesis_evaporate.py`)	Direct (Pair F1)	Code+ (Pair F1)	EVAPORATE-Hybrid (H_1)	Mejora (ΔF1)	Coste Code+	Coste Hybrid	Ahorro vs Direct
FDA 510(k) (PDF/TXT)	45.5	62.8	**71.0**	+8.2	$38	$91	31.7×
Enron Emails (TXT)	93.8	86.9	**90.2**	+3.3	$12	$19	22.7×
Wiki NBA (HTML)	44.8	68.2	**76.0**	+7.8	$60	$207	62.9×
SWDE Movie (HTML)	45.2	56.8	**69.0**	+12.2	$46	$155	36.5×
SWDE Univ. (HTML)	53.8	59.0	**68.8**	+9.8	$38	$102	37.2×
Promedio Global	**56.6**	**66.7**	**75.0**	**+8.3 pts**	**$39**	**$115**	**38.2×**

6. Relación con la asignatura

- **6.1. Conceptos que aparecen:** *Bloque I:* Datos no estructurados a accionables y caso FDA (`B1_1_4` , diaps. 6 y 14), Web Scraping con `BeautifulSoup` , `XPath` y `regex` (diaps. 20–32), ETL/ELT y transformaciones *Model-driven vs Data-driven* (`B2_2_2` , diaps. 9–27). *Bloque II:* Chunking, **TF-IDF + K-Means** y métricas Precision/Recall/F1 (`2_1_1` y `2_2_RAG`). *Bloque III:* Descubrimiento e integración de esquemas (`Block 3.1/3.2`).
- **6.2. Cómo amplía la asignatura:** Automatiza con LLMs la escritura manual de parsers de Web Scraping, resuelve la limitación de la diapositiva 14 de ETL/ELT (*"una función rara vez cubre el 100% de los casos"*) mediante Weak Supervision, y une los paradigmas *Data-driven* y *Model-driven*.